System design guide, part 2

Contents

Same idea as before. We sit together and work through each design question the way you would actually face it. For every question I first explain what the person asking is really trying to see in you, then how I would slowly build the design, clarifying the requirements first, doing a rough estimate where it helps, and only then deciding the pieces, giving the reason and the trade-off behind each choice. A high-level design for every question, and a low-level one where the internals matter, followed by the follow-up questions that usually come next, with short answers. The depth is pitched at a senior 2026 level.

1. Design a chat messaging system for 1:1 messaging (end-to-end). Discuss message delivery, ordering, presence, and read receipts.

What this question is really testing. Whether you can design a real-time system where clients are often offline, whether you get the four things the question names right (delivery, ordering, presence, read receipts), and whether you understand what “end-to-end” forces on the design, that the server only ever sees ciphertext and cannot read the messages. The usual follow-ups they keep ready are offline delivery, ordering guarantees, multi-device with end-to-end encryption, and presence at scale.

How I would drive it. Since this can sprawl quickly, I first pin the scope. It is 1:1 only, not groups. It is real-time, and the clients are mobile, so being offline is the normal case, not the exception. On delivery I would aim for at-least-once with de-duplication, so it feels exactly-once to the user. Ordering should be correct per conversation, not globally, since global ordering is expensive and nobody needs it. And we want presence (online and last-seen), read receipts, and end-to-end encryption, where the server stores and relays ciphertext but can never read the content.

Message delivery. The clients hold a persistent WebSocket to a connection gateway, and the gateway keeps a map of which user is on which connection. When A sends a message, it goes to the message service, which persists it and then, if B is online, pushes it down B’s WebSocket; if B is offline, it is stored and a push notification is sent, and B pulls it on reconnect. This store-and-forward is the heart of it, because the recipient is so often offline. For the “feels exactly-once” behaviour, the sender attaches a client-generated message id, so a retried send is de-duplicated rather than delivered twice.

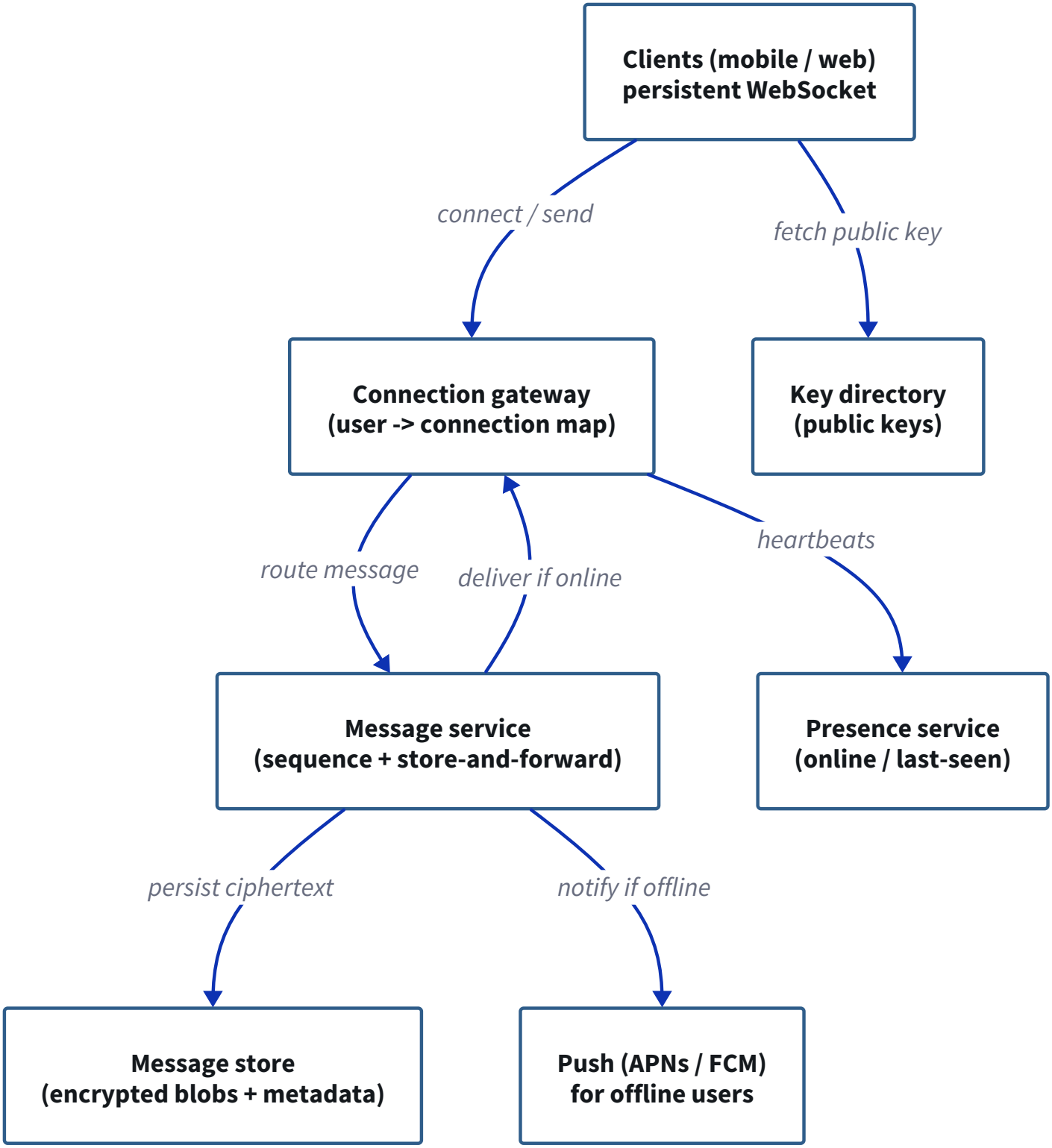
Ordering. The message service assigns a per-conversation sequence number as it persists each message. The client then orders by that sequence, so even if messages arrive out of order over the network, they are shown in the right order. I would keep this per conversation on purpose, because a global order across all conversations would need coordination that buys nothing for a 1:1 chat.

Presence. The clients send heartbeats over the same WebSocket, and a presence service tracks who is online and stores the last-seen time. When someone's presence changes, it is pushed out to the people who are chatting with them. Presence is best-effort by nature, since a dropped connection takes a moment to notice, and that is fine.

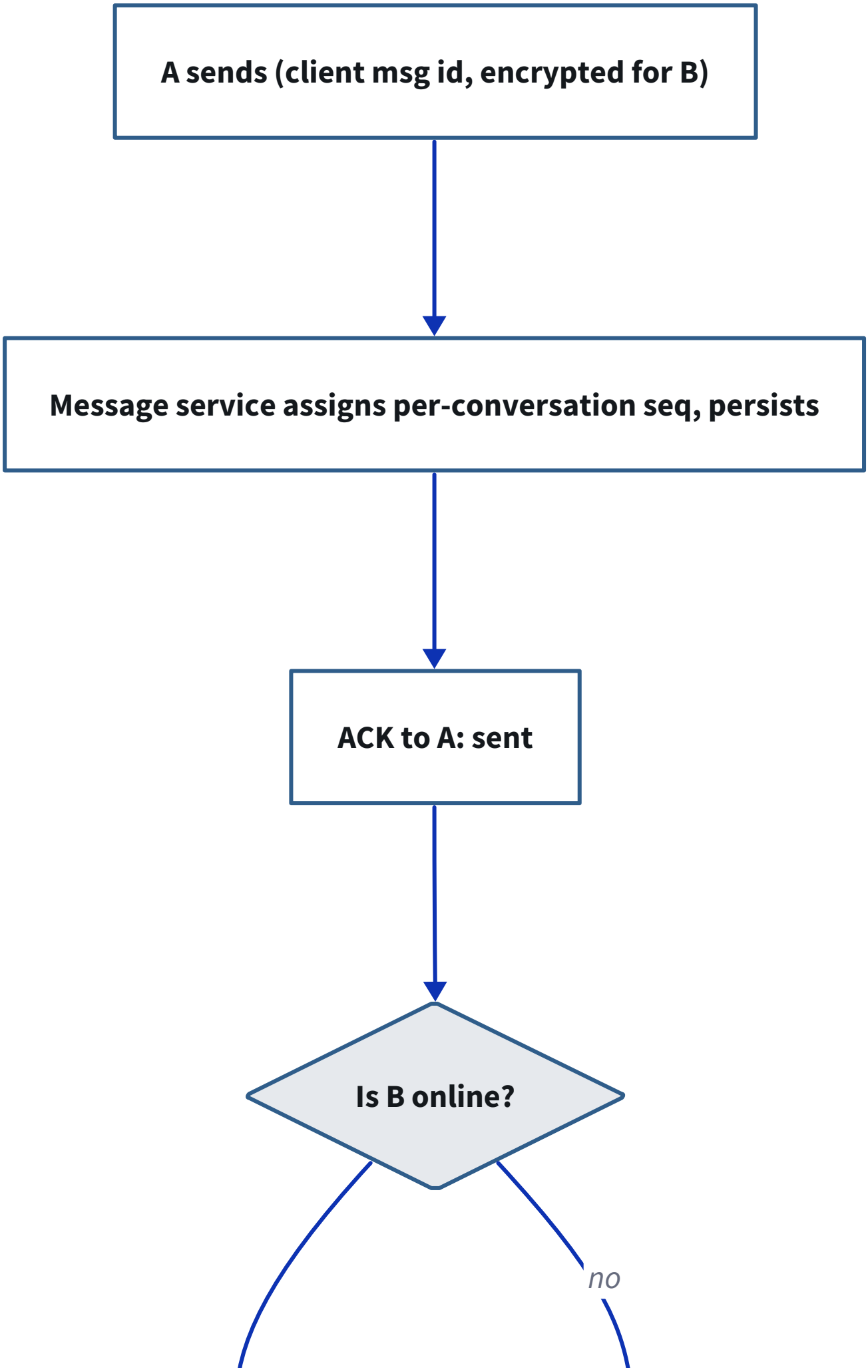
Read receipts. There are three states, and each is an event flowing back to the sender: sent (the server has accepted the message), delivered (the recipient's device has actually received it and acked), and read (the recipient has opened the chat). This is the single-tick, double-tick, blue-tick model. The nice part with end-to-end encryption is that these receipts ride on the metadata, so they keep working even though the server cannot read the content.

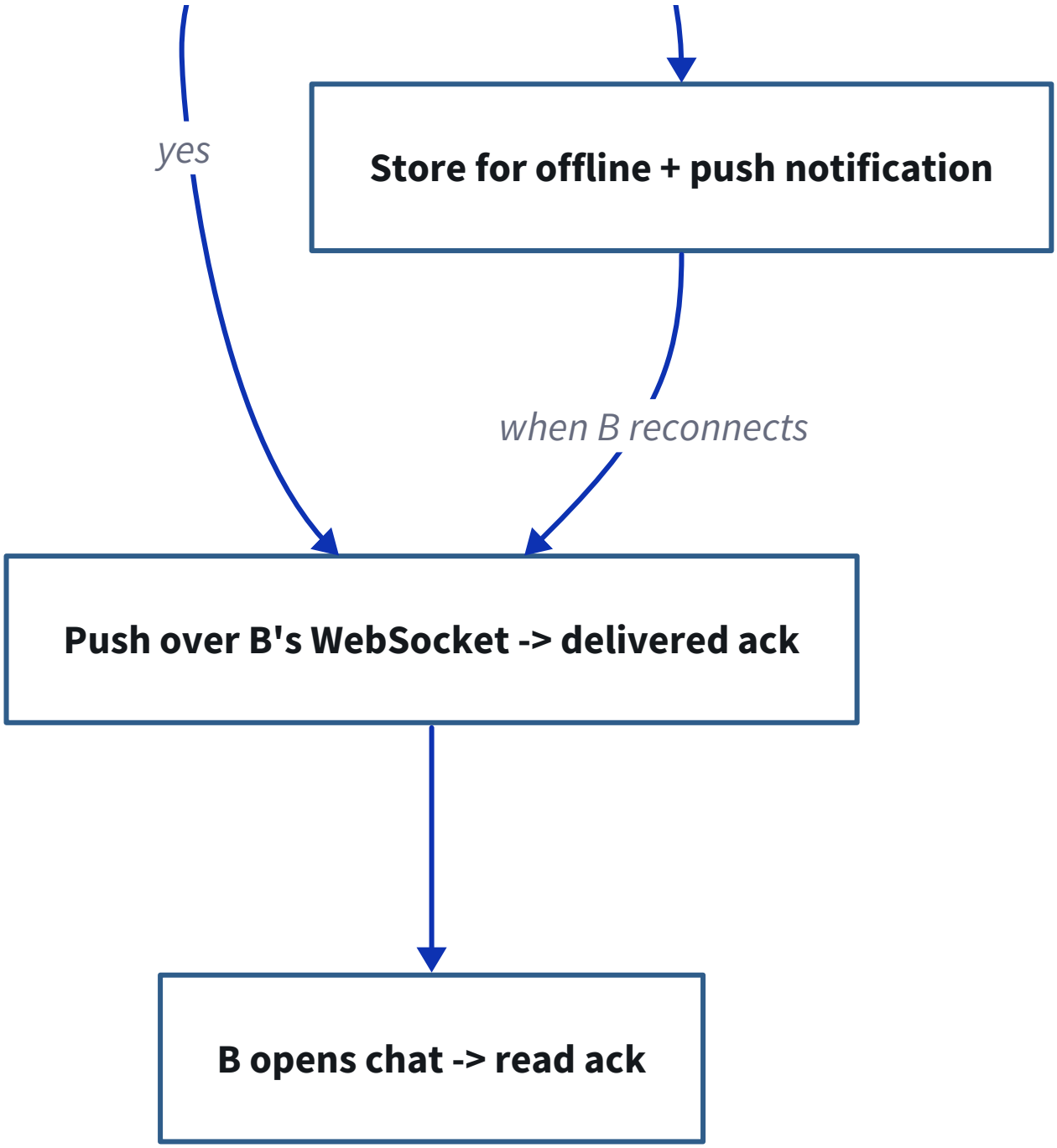
End-to-end encryption. The clients fetch each other's public keys from a key directory, and the sender encrypts the message with the recipient's key, so the server only ever stores and relays an encrypted blob plus the metadata (who, when, sequence), never the plaintext. In practice this is the Signal-style double ratchet, which also gives forward secrecy. The important design point to say out loud is that delivery, ordering, and receipts all run on metadata, so encryption does not break any of them.

High-level design.



Low-level design (send and deliver, with acks). A sends a message carrying a client message id and encrypted for B. The message service assigns the per-conversation sequence and persists the ciphertext, then acks “sent” back to A. If B is online, it is pushed down B’s socket and B’s device acks “delivered”; if B is offline, it is stored and a push notification goes out, and it is delivered when B reconnects. When B actually opens the chat, a “read” ack flows back.


A sends (client msg id, encrypted for B)
Message service assigns per-conversation seq, persists
ACK to A: sent
Is B online?
no



Follow-ups they will push on.

- **How do you avoid duplicate messages?** A client-generated message id, deduped server-side, so a retried send is not delivered twice.
- **Offline delivery?** Store-and-forward: persist for the offline user, push-notify, and deliver on reconnect.
- **Ordering guarantee?** A per-conversation sequence number; the client orders by it, so out-of-order arrival still displays correctly.
- **End-to-end with multiple devices?** Each device has its own key, so the sender encrypts once per recipient device.
- **Does encryption break read receipts?** No, receipts and ordering ride on metadata, not the encrypted content.
- **Presence at scale?** Heartbeats plus a presence service, fanned out only to a user's active conversations, and treated as best-effort.

2. Design a CI/CD system for microservices. Cover build artifacts, environment promotion, canary/blue-green deployments, and rollback mechanics.

What this question is really testing. Whether you understand the core discipline that makes microservice delivery safe: build an immutable artifact once, promote that same artifact through environments, roll out progressively with canary or blue-green, and roll back fast and deterministically. The usual follow-ups they keep ready are why you build only once, database migrations during a rollback, and how you pick between canary and blue-green.

How I would drive it. Let me settle the shape first. We have many microservices, each one independently deployable, and the goal is fast but safe deploys through the usual environments, dev to staging to production. We want progressive delivery in production and a rollback that is quick and reliable. With that fixed, the pipeline almost designs itself.

Build artifacts. On every commit, CI builds the service, runs the tests, and produces an immutable artifact, in practice a container image, tagged with the commit SHA and pushed to a registry. The key word is immutable: the artifact is built once and never rebuilt, so what you tested is exactly what runs in production.

Environment promotion. The same image is then promoted through the environments, dev to staging to production, and it is never rebuilt per environment. Only the configuration is injected per environment, following the twelve-factor idea of config outside the artifact. This is what guarantees that a bug cannot creep in between staging and production, since the bytes are identical; only the config differs.

Canary and blue-green deployments. Two progressive strategies, and they trade off differently:

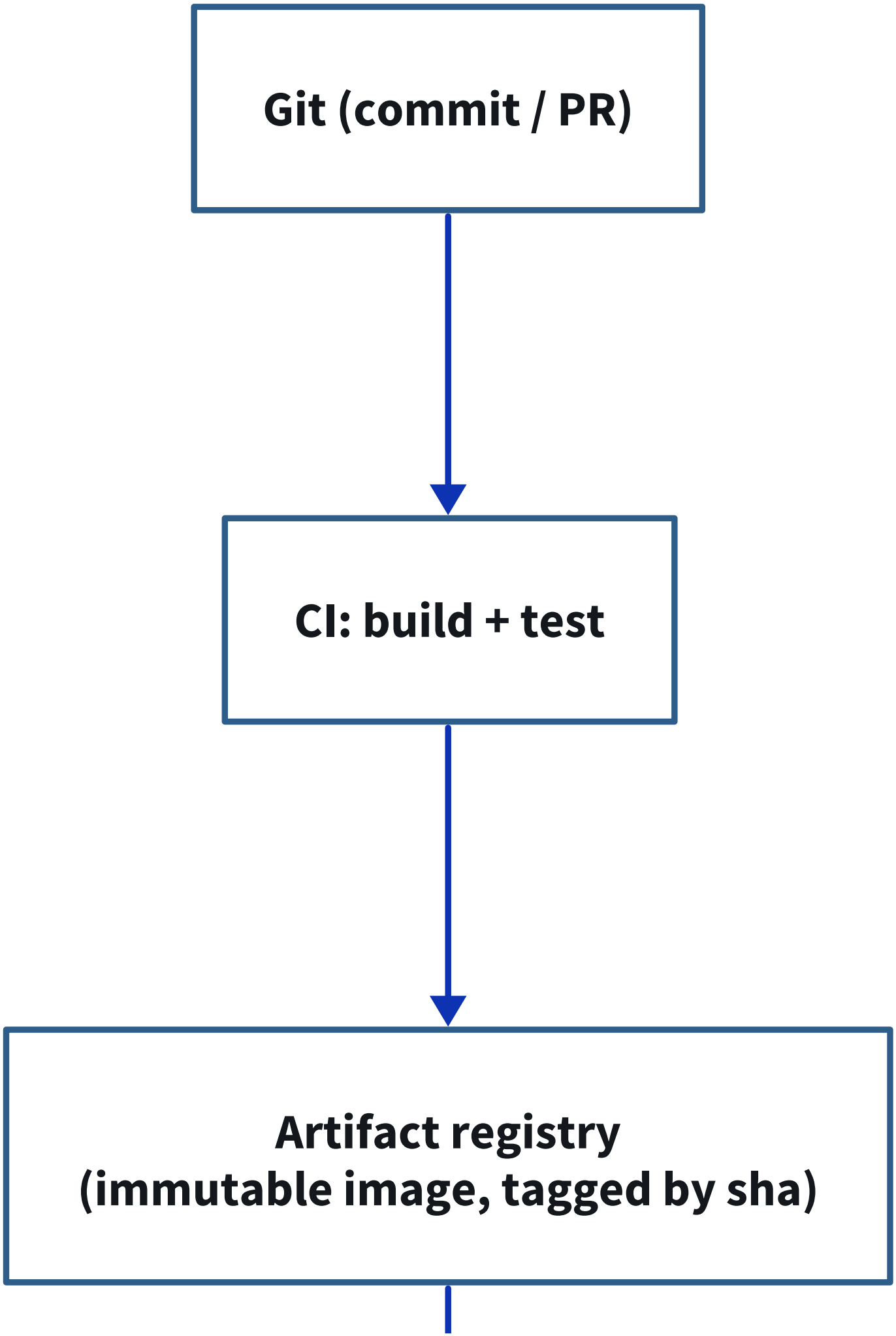
	Blue-green	Canary
How it works	Two identical prod environments; deploy to the idle one, then switch all traffic	Route a small % of traffic to the new version, then ramp up gradually

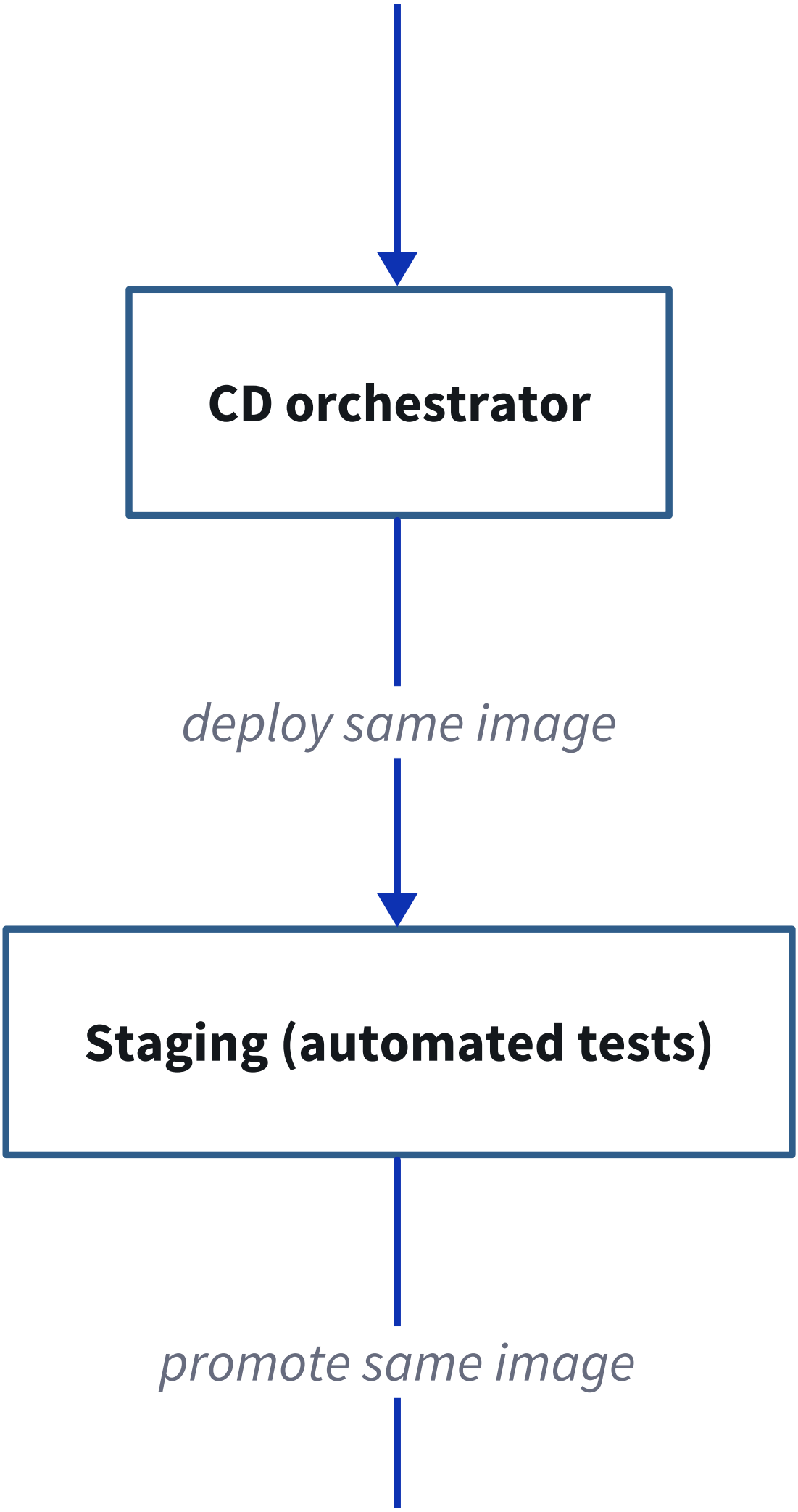
	Blue-green	Canary
Rollback	Instant, switch traffic back to the old environment	Route the small % back to the old version
Risk exposure	All users switch at once	Only a small slice is exposed while you watch
Cost	Roughly double the infra during the switch	Cheaper, runs both versions but not a full second copy

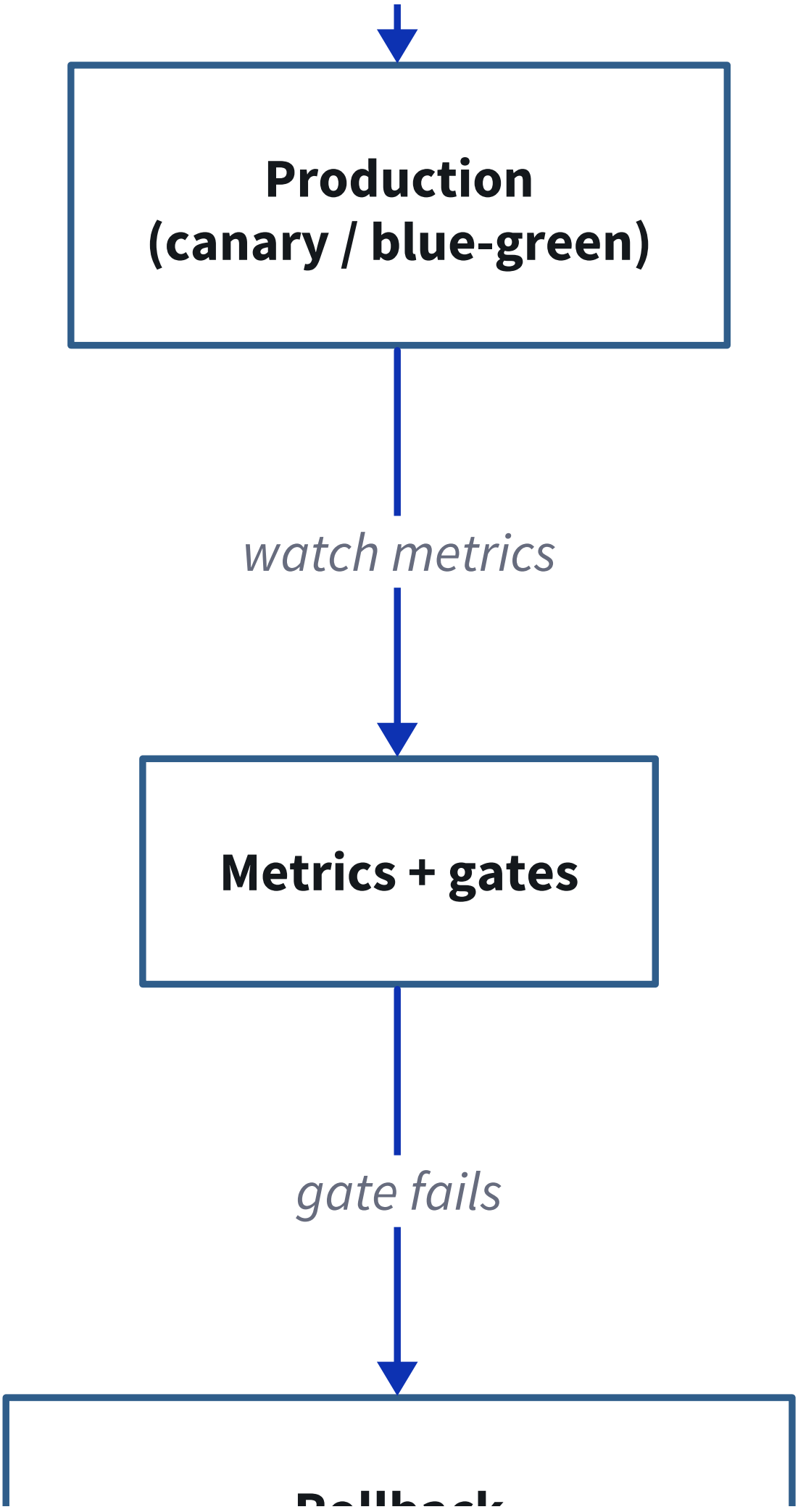
So blue-green gives you an instant, clean switch at the cost of running two full environments, while canary limits the blast radius by exposing only a slice first. For most services I would lean canary, since watching a small slice before committing is the safer default.

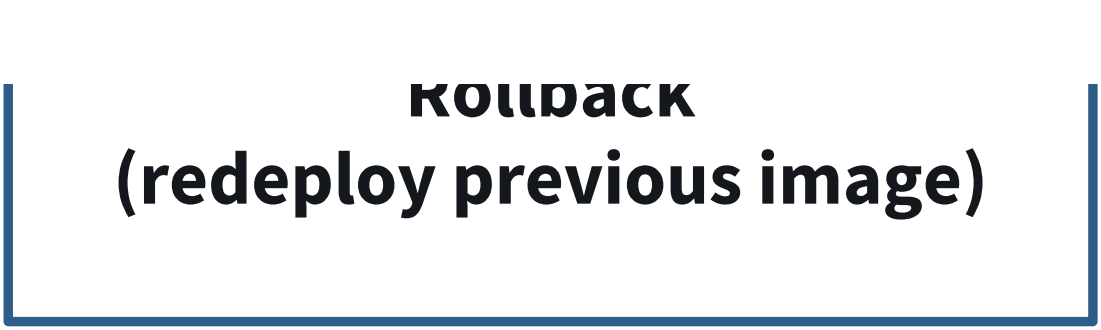
Rollback mechanics. Because the artifacts are immutable and versioned, a rollback is simply redeploying the previous image, or shifting traffic back to the old version, which is fast and deterministic. The one genuinely hard part is database migrations: a rollback of the code does not undo a schema change, so you keep migrations backward-compatible (expand-then-contract) so that the previous version of the code still runs against the new schema.

High-level design.

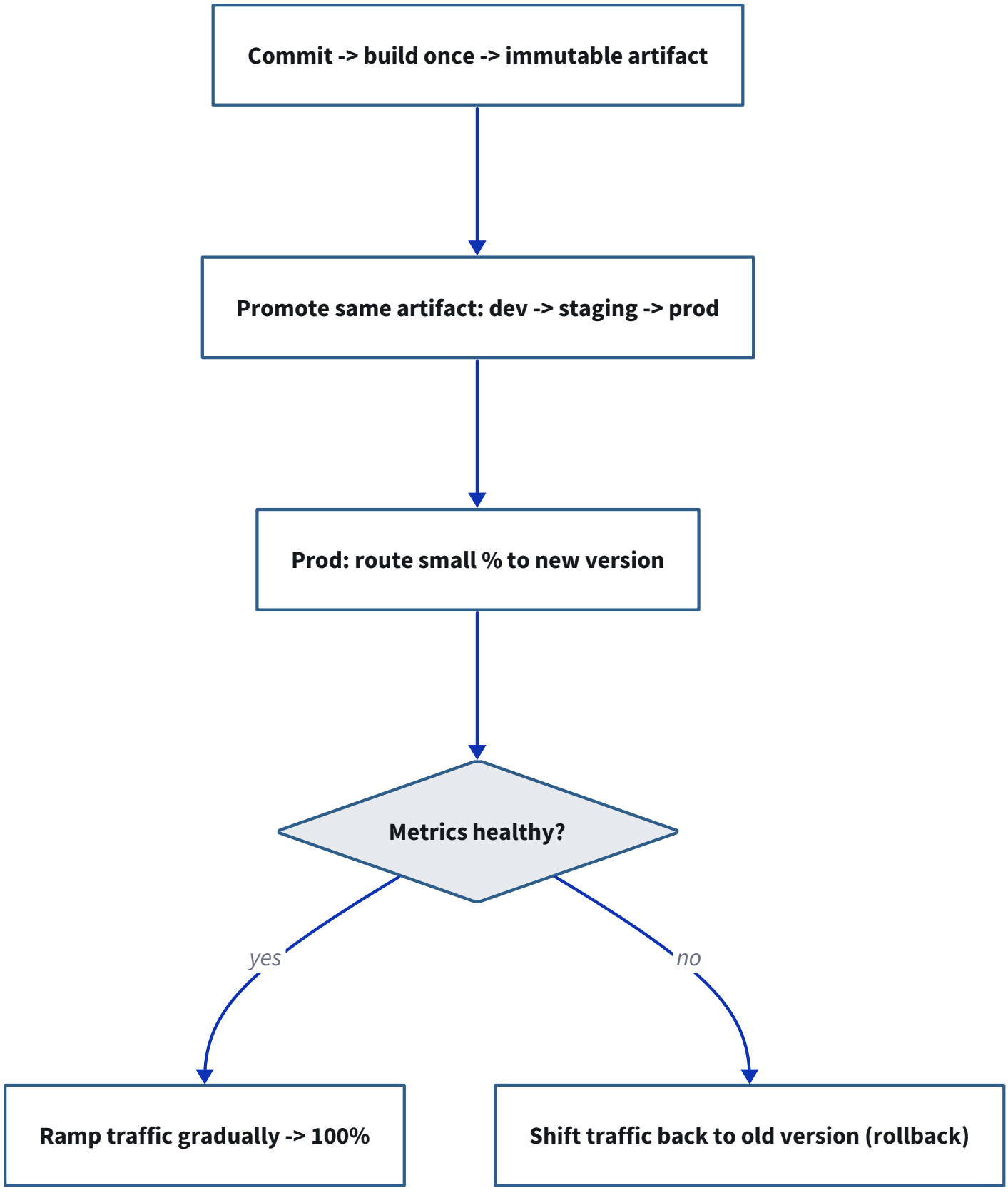

Git (commit / PR)
CI: build + test
Artifact registry
(immutable image, tagged by sha)


CD orchestrator
deploy same image
Staging (automated tests)
promote same image


Production
(canary / blue-green)
watch metrics
Metrics + gates
gate fails



Low-level design (the pipeline). A commit triggers a build-and-test that produces one immutable artifact. That same artifact is promoted, dev to staging to production, with automated tests gating each step. In production it goes out as a canary, a small percentage first, and an automated metric gate decides whether to ramp up gradually to 100% or to shift traffic back to the old version as a rollback.



Follow-ups they will push on.

- **Why build only once?** So the artifact you tested is byte-for-byte the one you ship; rebuilding per environment can introduce differences.
- **Rollback with a database migration?** Keep migrations backward-compatible (expand-then-contract) so the old code still runs against the new schema.

- **Canary or blue-green?** Canary limits the blast radius and costs less; blue-green gives an instant clean switch at double the infra.
- **How does config differ per environment?** Inject it at deploy time (twelve-factor), keep it out of the artifact.
- **Secrets?** From a secrets manager at runtime, never baked into the image.
- **Automated rollback?** Tie the metric gate to error rate and latency SLOs so a breach rolls back without a human.

3. Design a canary deployment strategy for a new model version. Hint: rollout strategy, metrics to watch, rollback criteria.

What this question is really testing. Whether you understand that rolling out a machine-learning model is not the same as rolling out code. The traffic mechanics look like an ordinary canary, but the metrics that decide success are model-quality and business metrics, not just latency and errors, and the rollback criteria must be pre-committed. The hint spells out the three parts, so I would structure the answer exactly around them: rollout strategy, metrics to watch, and rollback criteria.

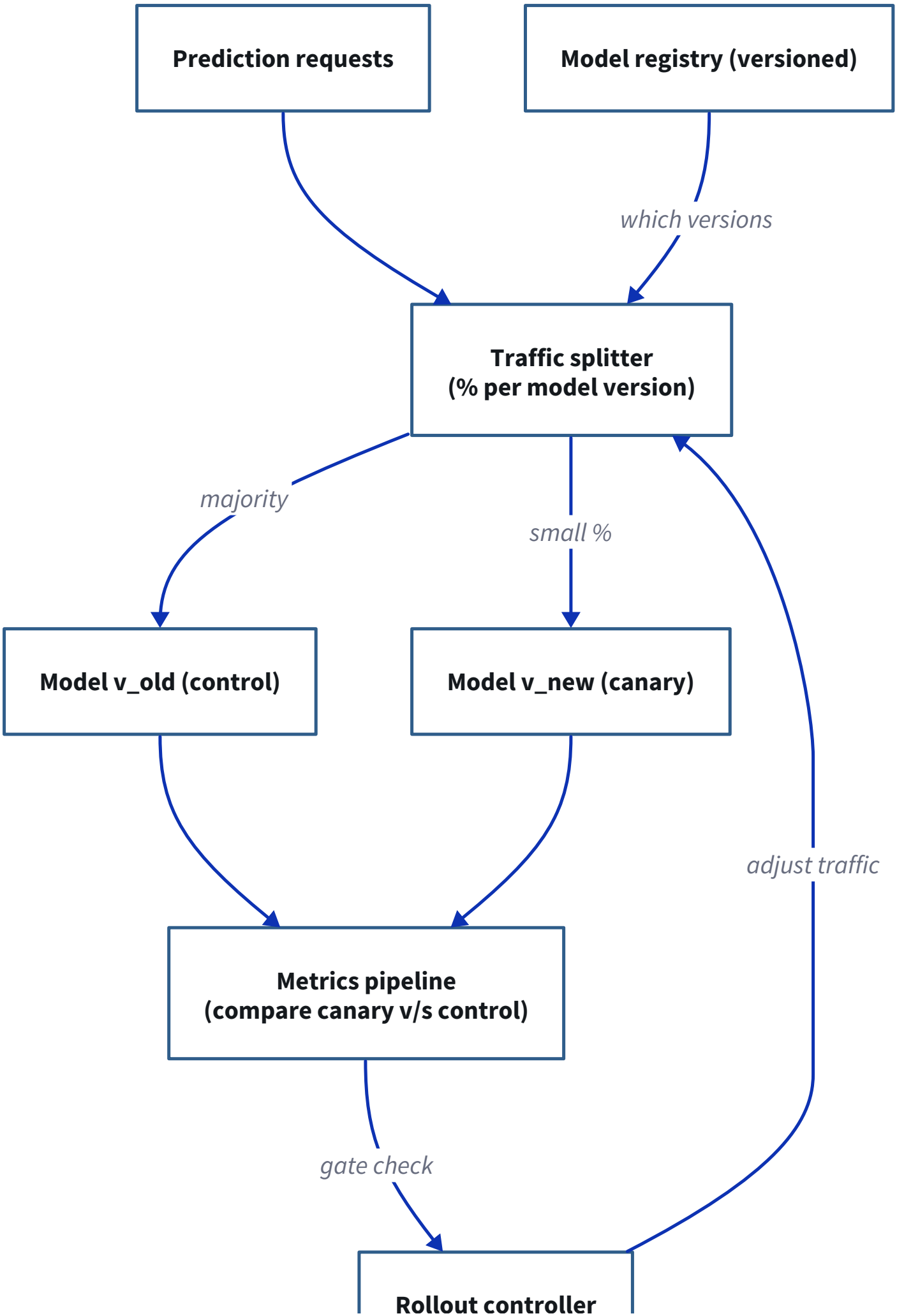
How I would drive it. Let me clarify the setting. There is a new version of a model, say a ranking or a fraud model, served behind a prediction API, and offline evaluation has already passed, so now we want to prove it online without hurting quality. The difference from a code deploy is that “healthy” here is not only “no errors and fast”, it is “the predictions are at least as good for the business”, and some of those signals arrive with a delay.

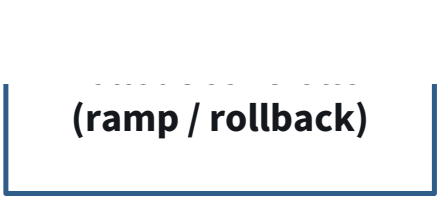
Rollout strategy. I would go in stages. First a shadow phase: mirror live traffic to the new model but do not serve its output, just log it and compare against the current model, so you catch obvious regressions with zero user risk. Then a canary: serve the new model to a small slice of users, 1% to start, with the old model as the control, and ramp gradually, 1% to 5% to 25% to 100%, with a gate at each step. Running the old and new side by side as control and treatment is really an A/B comparison, which is what lets you attribute any change to the model and not to the day’s traffic.

Metrics to watch. Two layers, and the second is the point. The operational layer is the usual latency (p99), error rate, and resource or cost, since a new model can be slower or larger. The model-quality and business layer is what actually decides it: the online outcome metrics like click-through, conversion, or fraud caught, plus guardrail metrics that must not regress (a key business KPI, fairness, or a safety metric). I would also watch prediction drift and calibration. The honest complication to raise is that some of these metrics lag, a conversion may take days, so the canary has to run long enough for the signal to be real.

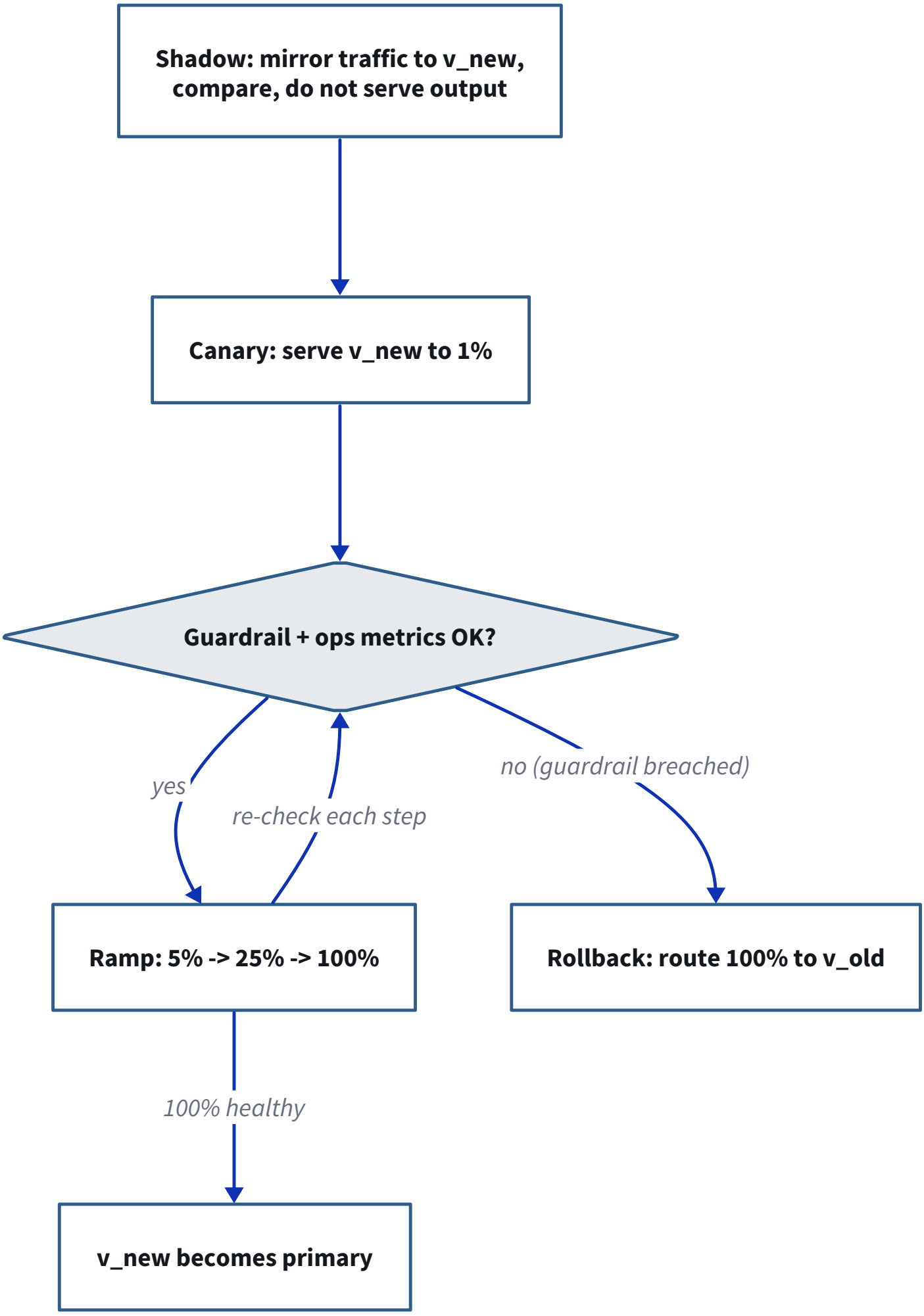
Rollback criteria. These are defined before the rollout, not judged by gut feel in the moment. If a guardrail metric drops beyond its threshold, or an operational SLO (latency or error rate) is breached, or a business KPI regresses with statistical significance, the controller routes 100% of traffic back to the old model automatically. Pre-committing the thresholds is what keeps a rollout honest, because otherwise there is always a temptation to explain away a bad number.

High-level design.


Prediction requests
Model registry (versioned)
which versions
Traffic splitter
(% per model version)
majority
small %
Model v_old (control)
Model v_new (canary)
adjust traffic
Metrics pipeline
(compare canary v/s control)
gate check
Rollout controller



Low-level design (the rollout state machine). It runs as a gated progression. Shadow first, mirroring traffic to the new model and comparing without serving it. Then canary at 1%, and at each step a gate checks the guardrail and operational metrics: if they hold, ramp to the next level (5%, 25%, 100%), re-checking at each step; if a guardrail is breached, roll back to 100% old model. Only a fully healthy ramp promotes the new model to primary.


Shadow: mirror traffic to v_new, compare, do not serve output
Canary: serve v_new to 1%
Guardrail + ops metrics OK?
yes
re-check each step
no (guardrail breached)
Ramp: 5% -> 25% -> 100%
Rollback: route 100% to v_old
100% healthy
v_new becomes primary

Follow-ups they will push on.

- **Shadow v/s canary?** Shadow compares with zero user risk but no real user feedback; canary serves real users to a small slice and measures the actual outcome.
- **Why not just trust offline evaluation?** Offline metrics miss real traffic, feedback loops, and business impact; online is where it is proven.
- **How long do you run the canary?** Long enough for the outcome metric to reach statistical significance, which for lagging metrics like conversion can be days.
- **What are guardrail metrics?** Business or safety KPIs that must not regress even if the headline metric improves.
- **A/B test v/s canary?** They overlap here; the canary is an A/B comparison with a gradual, safety-gated traffic ramp.
- **Delayed feedback?** Gate the fast operational metrics immediately, and hold the ramp until the slower business metrics are in.